Week 5 Assignment Submission

Name: Marie Lynott

Date: 6/27/26

What I Accomplished This Week

- Updated the test_gemini() function in rag_app.py to make a real call to the Gemini API.
- Successfully connected the backend to Google's Gemini model.
- Tested the /test-gemini endpoint, which now returns a generated response from Gemini. ("status":"success","prompt":"Explain what a large language model is in one short paragraph.","response":"A large language model (LLM) is an artificial intelligence system trained on vast amounts of text data to understand, translate, and generate human-like language. By using deep learning algorithms to analyze patterns across millions of books, articles, and websites, the model learns how words and phrases relate to one another. This enables it to perform a wide variety of tasks—such as answering questions, writing essays, and generating code—by predicting the most likely next word in a sequence.")
- Learned how to load the API key securely from the .env file.
- Committed and pushed the changes to GitHub.

Challenges I Faced

The biggest challenge was finding the correct model name. I had to try several different model names (gemini-1.5-flash, gemini-1.5-flash-latest, gemini-pro, gemini-2.5-flash, etc.) before the API call worked. This took multiple attempts and restarts of the server. I learned that model names can change and it's important to check the current valid names in the Gemini documentation.

What I Learned

What I Learned

- **How to make an API call to a Large Language Model (Gemini) from Python code.** I learned the basic pattern for calling an LLM: First, you configure the client using your API key. Then you create a model object (in this case using genai.GenerativeModel()). Next, you send a prompt to the model using the generate_content() method. Finally, you extract the text response from the object that Gemini returns. This is the fundamental flow for interacting with Gemini and most other LLM APIs.
- **The importance of keeping the API key secure in a .env file and loading it with python-dotenv.** This keeps sensitive information (the secret key) out of the actual Python code. Instead of hardcoding the key directly in rag_app.py (which would be dangerous if the code is ever shared or uploaded to GitHub), we store it in a separate

.env file. The load_dotenv() function then safely brings that key into the program when it runs. This is a standard industry practice for handling secrets securely.

- **How FastAPI endpoints work (the /test-gemini route returns JSON with the AI's response).** I now understand that FastAPI allows us to create web routes (endpoints). When someone visits a URL like http://127.0.0.1:8000/test-gemini, the server runs the test_gemini() function in the background. That function calls Gemini, gets a response, and sends the result back in JSON format. The browser then displays that JSON. This structure is how real backend applications expose AI functionality to the front end.
- **That working with AI APIs requires patience and debugging — especially when dealing with model names and configuration.** Even though I followed the assignment instructions, I still ran into multiple errors related to the model name. I had to try several different model names (gemini-1.5-flash, gemini-pro, gemini-2.5-flash, etc.) and restart the server each time before it worked. This taught me that real development is rarely straightforward. You often have to read documentation, experiment, and troubleshoot issues even when you think you did everything correctly.

Key Takeaway

Even though I followed the assignment steps, I had to experiment and troubleshoot to get the Gemini call working. This showed me that in real projects, you often need to test and adjust things multiple times before they work correctly. I now have a better understanding of how backend AI calls are structured, even though I'm still new to coding. This experience also reinforced the importance of patience and persistence when working with AI tools and APIs.

GitHub Commit Message:

"Week 5: Implemented first Gemini API call in /test-gemini endpoint after troubleshooting model names"